



Treinamento em Segurança Ofensiva 2024/2025

Relatório de Atividades	
Módulo 4:	Pentest em Aplicações Web
Atividade 10:	GraphQL
Nome:	Luiz Eduardo de Andrade

1. Introdução

Estas atividades fazem parte do módulo *Pentest em Aplicações Web*, para a qual utilizamos a sala do PortSwigger - <https://portswigger.net/>

No contexto destas salas, nosso foco é:

- 1) Acessando postagens privadas do GraphQL, a vulnerabilidade é utilizar intstruções inql para descobrir a senha e o impacto é que depois que ele passar da tela de login terá acesso à sua aplicação com os mesmos níveis que o usuário que ele roubou as credenciais;
- 2) Conseguir acessar através de postagens privadas o password do usuário;
- 3) Em Requests copiar a URL no Burp ir na guia Inql Scanner para acessar as queries do GraphQL;
- 4) Forçar um erro de senhas incorretas tanto no Burp quanto na página da internet.
- 5) Utilizar força bruta com uma lista de senhas através de script Python e utilizando também o Burp com a extensão inql e através desta lista de passwords encontrar o password válido.
- 6) Ignorando proteções de força bruta do GraphQL a vulnerabilidade é conseguir atravessar a fase de login com uma lista de palavras para a senha e o impacto é com o usuário e senha pode entrar na sua aplicação sem passar corretamente pela etapa de login e ter acesso a ela.
- 7) Instalar a extensão inql no Burp.
- 8) Forçar um erro de senhas tentadas para bloquear o acesso
- 9) Fazer um script Python para testar login e senha até achar o certo.

- 1) Acessando postagens privadas do GraphQL- <https://portswigger.net/web-security/graphql/lab-graphql-reading-private-posts>
- 2) Ignorando proteções de força bruta do GraphQL- <https://portswigger.net/web-security/graphql/lab-graphql-brute-force-protection-bypass>

2. Desenvolvimento

2.1. Configuração do ambiente

Os seguintes passos foram necessários para configurar o ambiente necessário para o desenvolvimento das atividades:

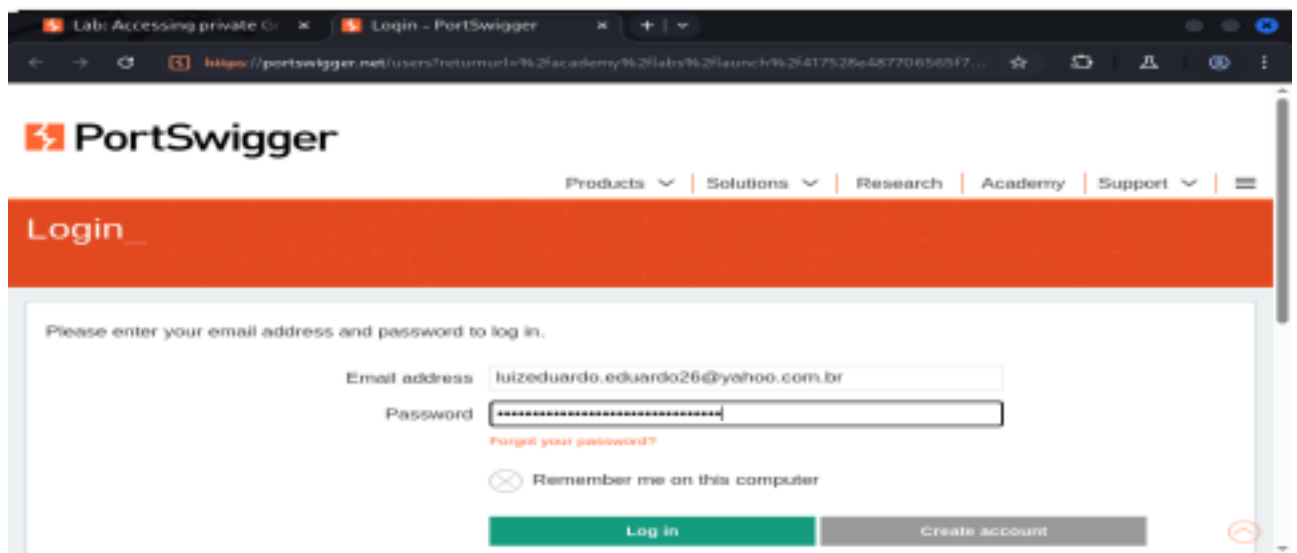
- 1) Acessar os sites informados acima.
- 2) Utilizar a VM do Kali Linux: Para utilizar o Burp e executar alguns comandos no terminal.

2.2. Atividade prática

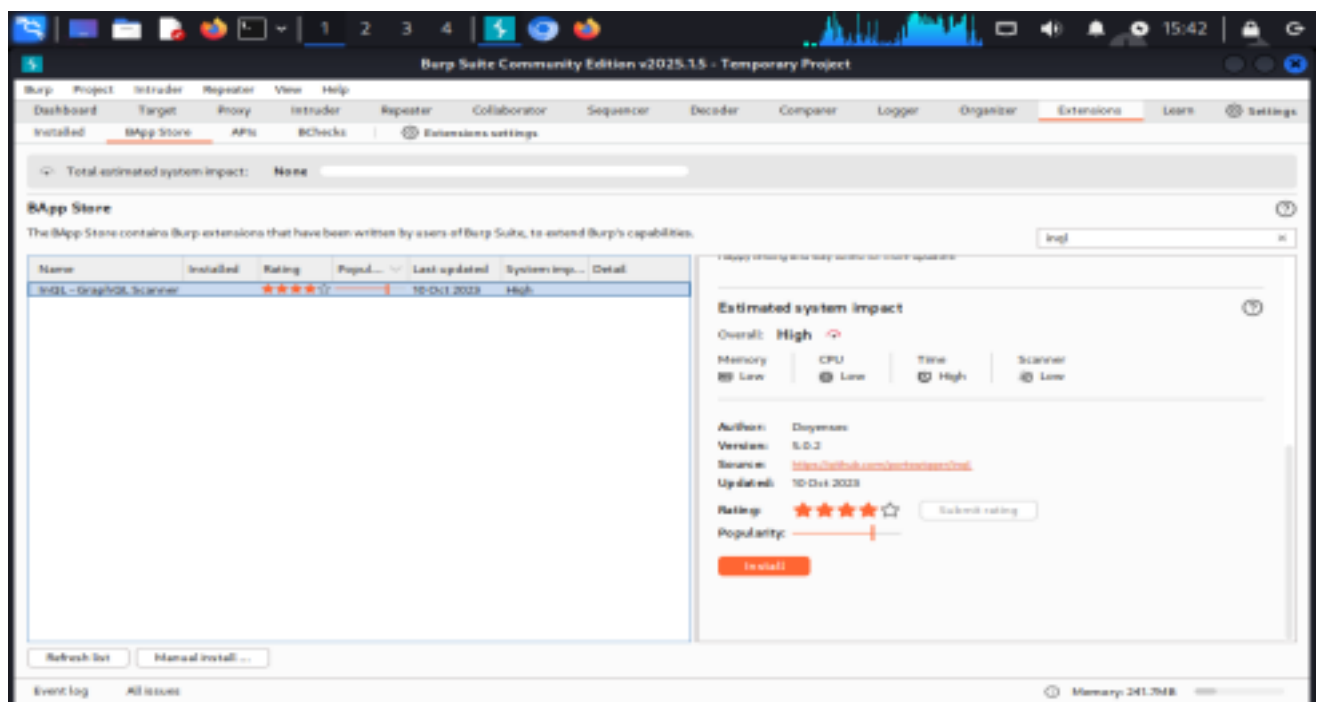
O objetivo desta atividade é realizar as 2 tarefas propostas no PortSwigger.

Atividade 1 - Acessando postagens privadas do GraphQL.

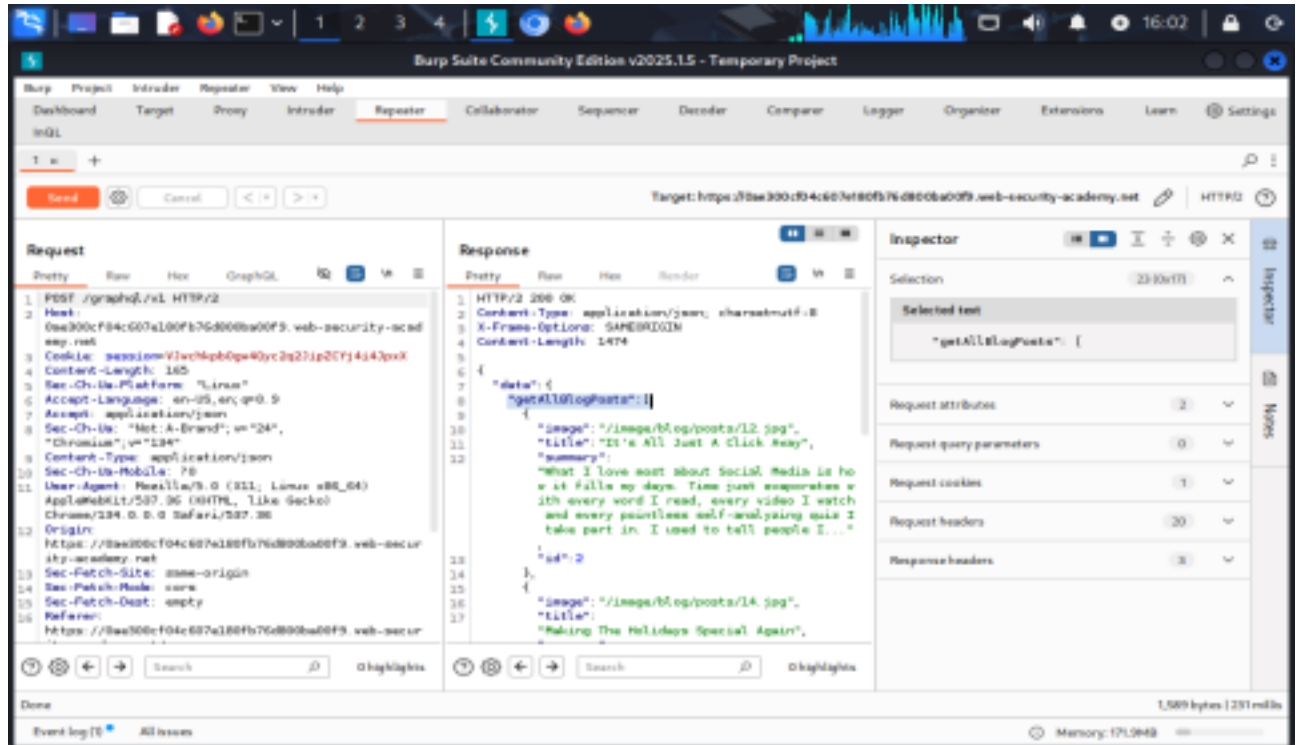
Passo 1 – Entre com seu login e senha no PortSwigger no site: - <https://portswigger.net/web-security/graphql/lab-graphql-reading-private-posts> e informe e-mail e senha.



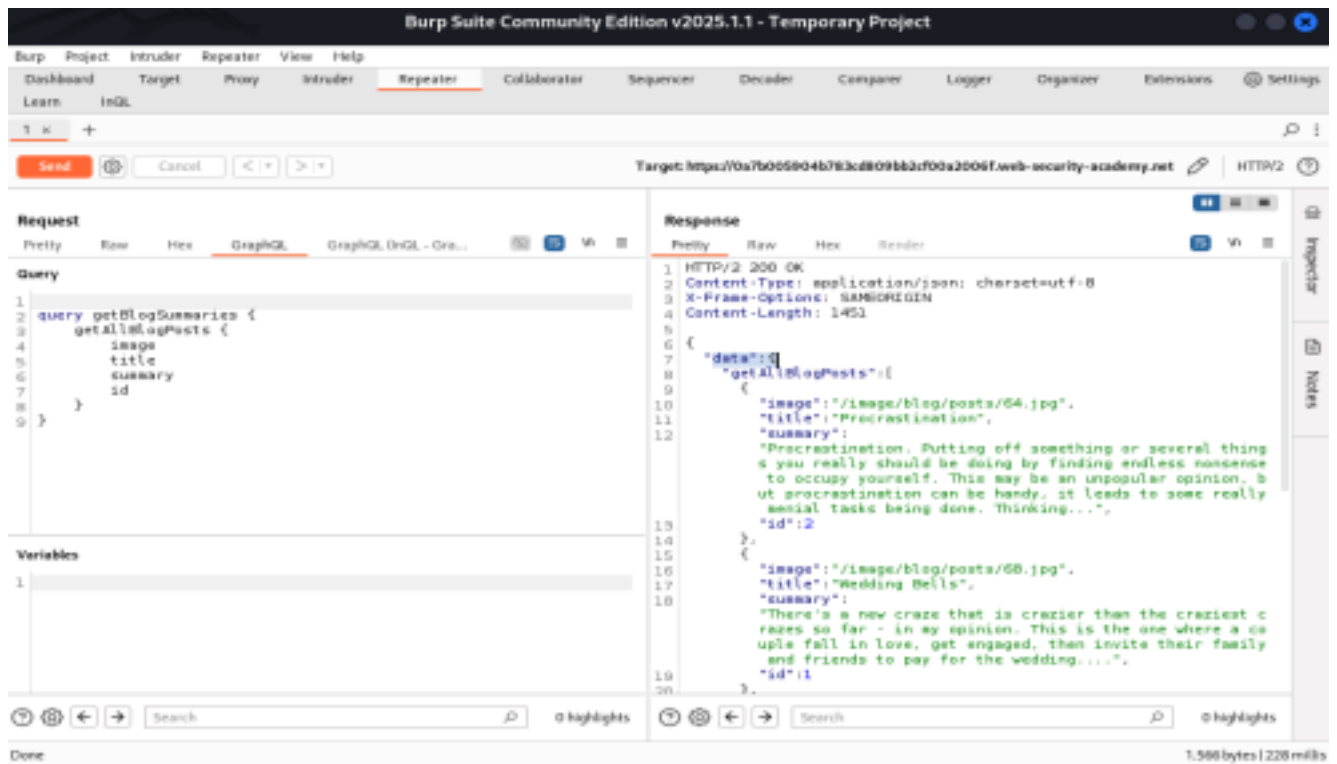
Passo 2 – No Burp vamos em Extensions, em BApp Store digitamos na barra de pesquisa inql e clicamos em instalar.



Passo 3 – Esta instalação é um pouco pesada tem impacto no sistema de nível médio, quando não precisar pode apagar indo na guia Installed e reinstalar quando precisar.



Passo 6 – Depois vamos em GraphQL e vemos a estrutura da query e verificamos no Response que não temos o id 3 então provavelmente é a postagem que estamos procurando.

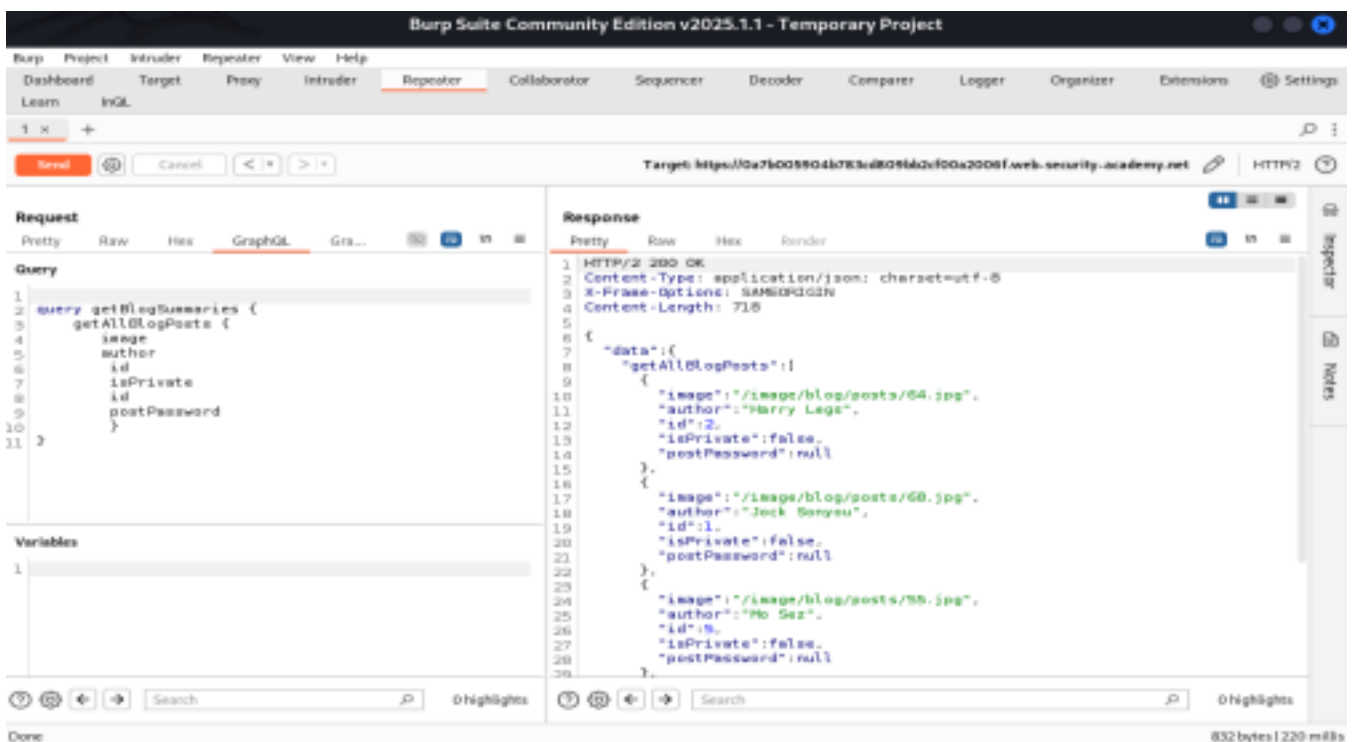


5

Passo 7 – Voltamos ao Pretty em Request e clique com o botão direito e copie a URL e depois vá até a guia InQL Scanner e cole na guia Provide URL of the GraphQL Endpoint, clique em Analyze e do lado esquerdo aparece duas consultas `getallblogpost` e `getblog post`.

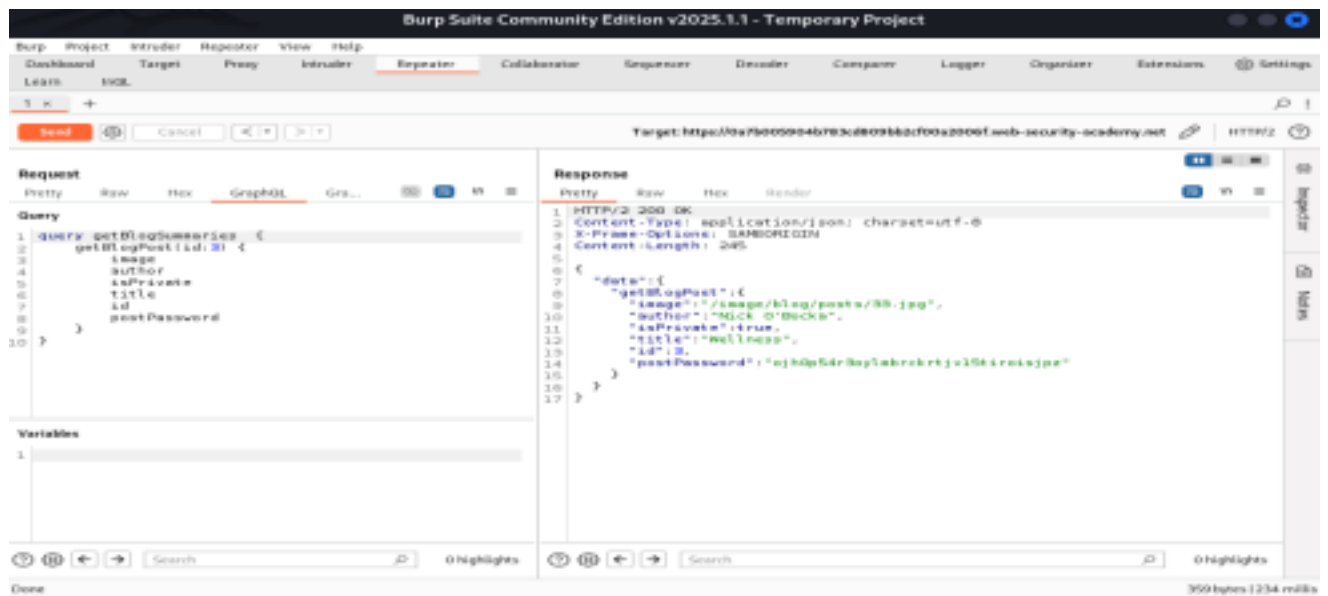


Passo 8 – Depois selecionamos a consulta getAllBlogposts copiamos a query, clicamos em Send e conseguimos maiores informações do que tínhamos anteriormente, refinamos a pesquisa e ainda não achamos o id 3.

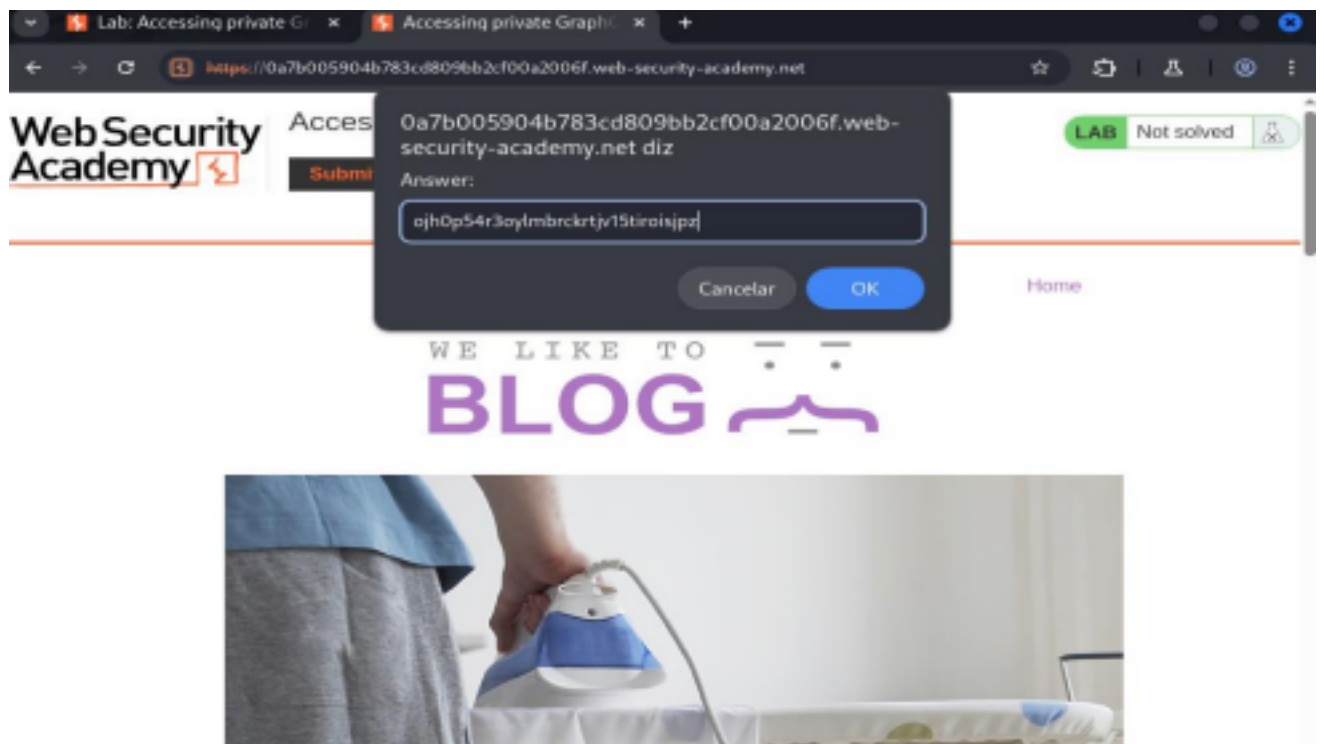


6

Passo 9 – Como verificamos que todas as mensagens retornadas estavam com o parâmetro isPrivate: false, então voltamos ao InQL Scanner e agora copiamos ele e colamos no nosso repeater, tiramos algumas informações que não são importantes como paragraphs, summary e date e colocamos no campo getBlogPost(id:3) que é o id que não aparecia nos resultados, clicamos em Send e conseguimos o password no campo postpassword.

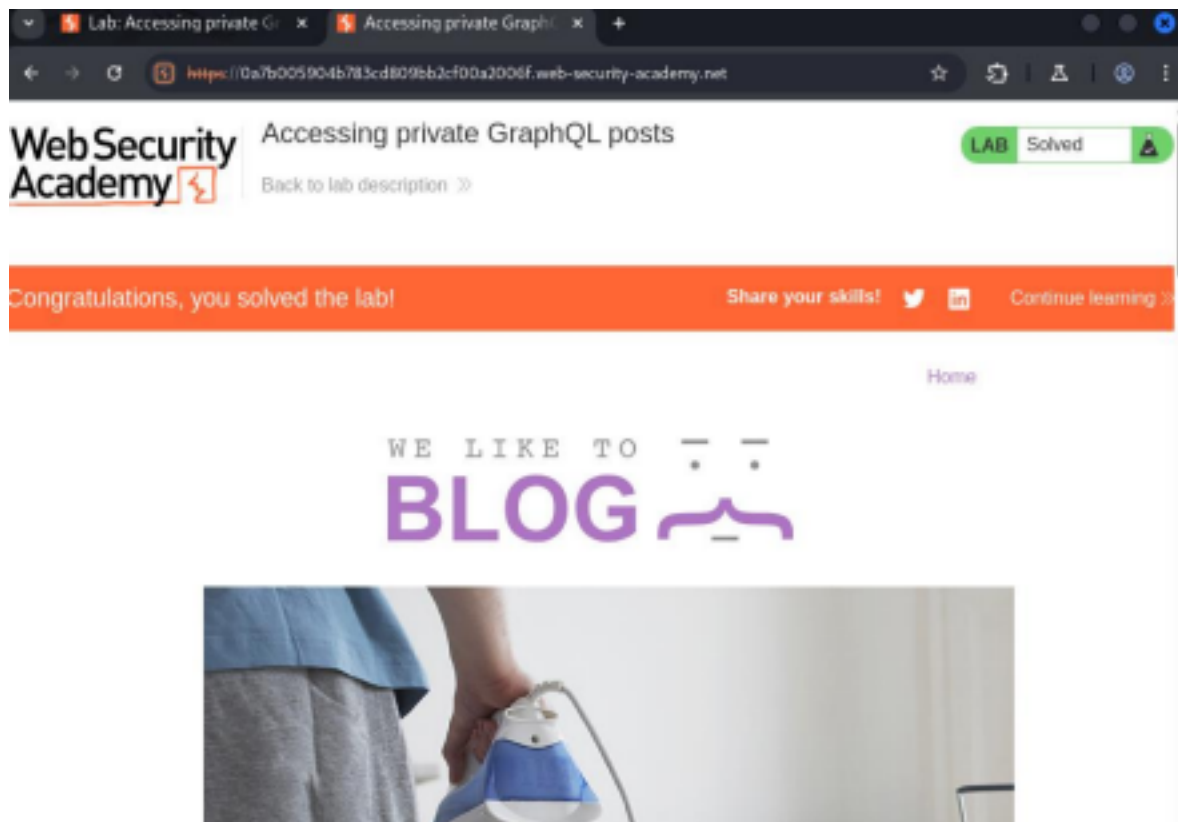


Passo 10 – Colamos o password na guia Submit Solution.



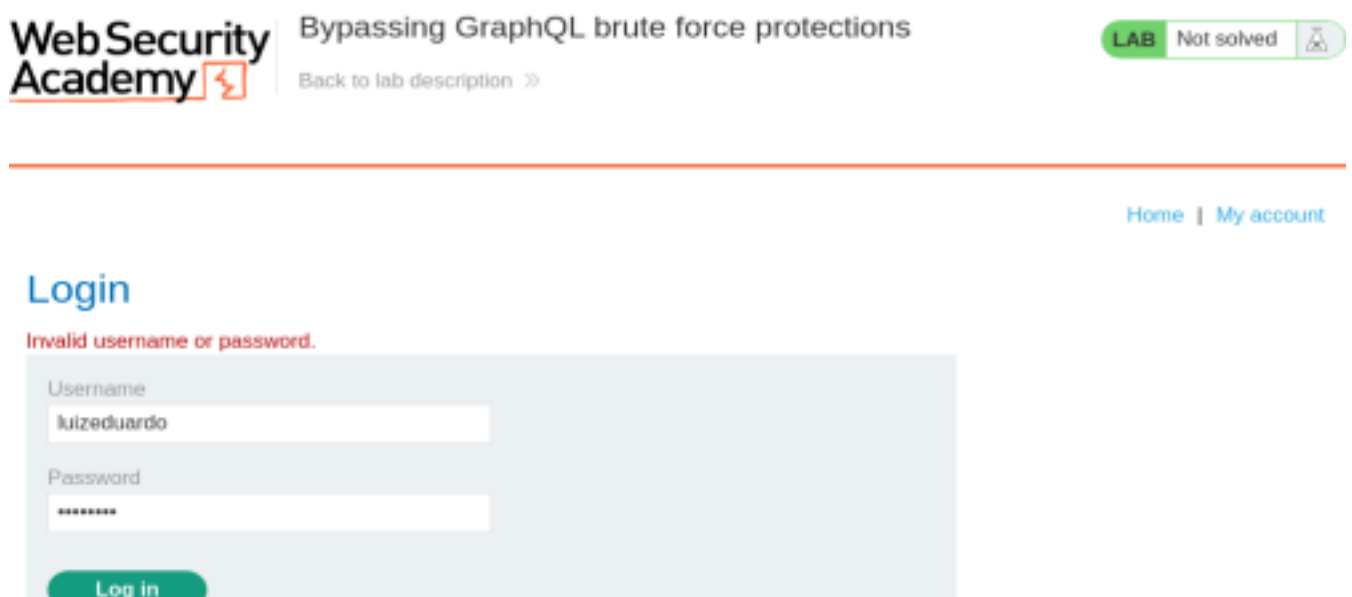
7

Passo 11 – A tarefa foi concluída com sucesso.



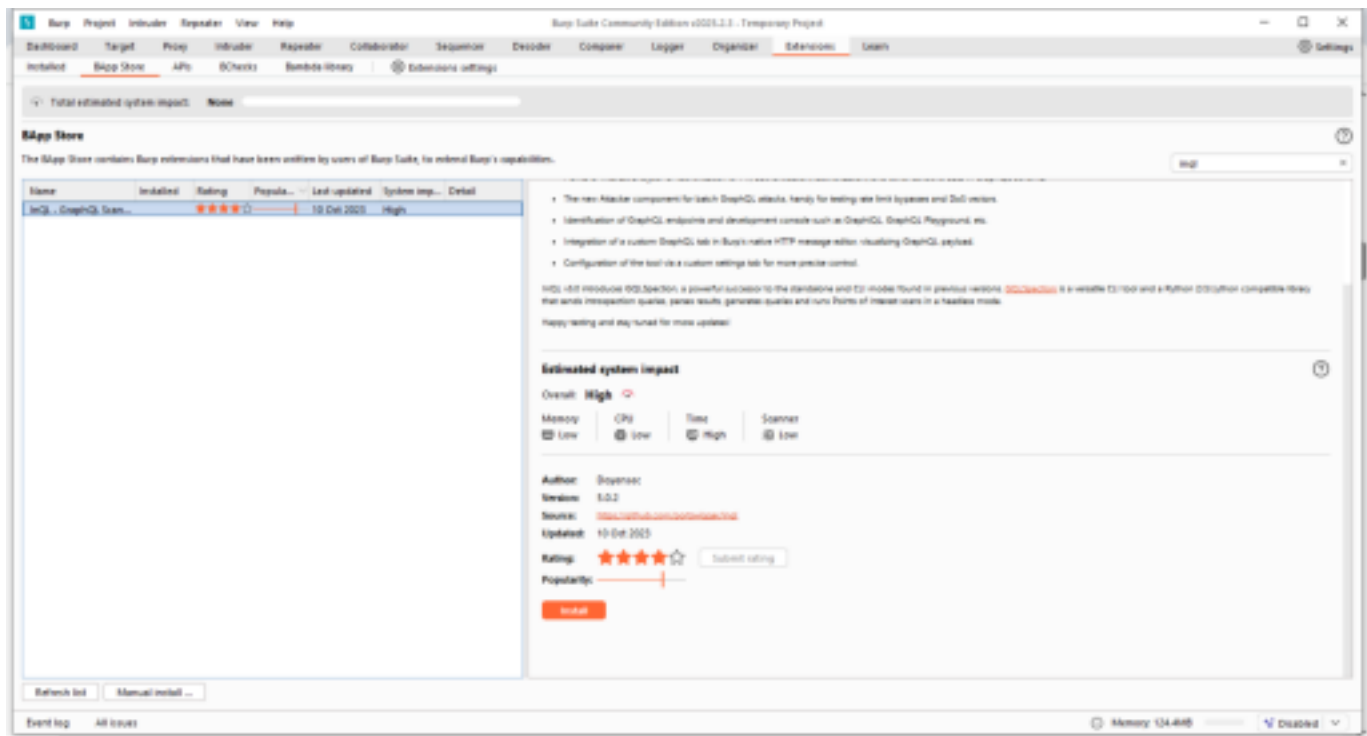
2.3. Atividade 2 – Ignorando proteções de força bruta do GraphQL.

Passo 1 – Acesse o laboratório clique em myaccount e entre com um usuário e senha inválidos, clique em login, irá aparecer na tela que não existe o usuário e password.

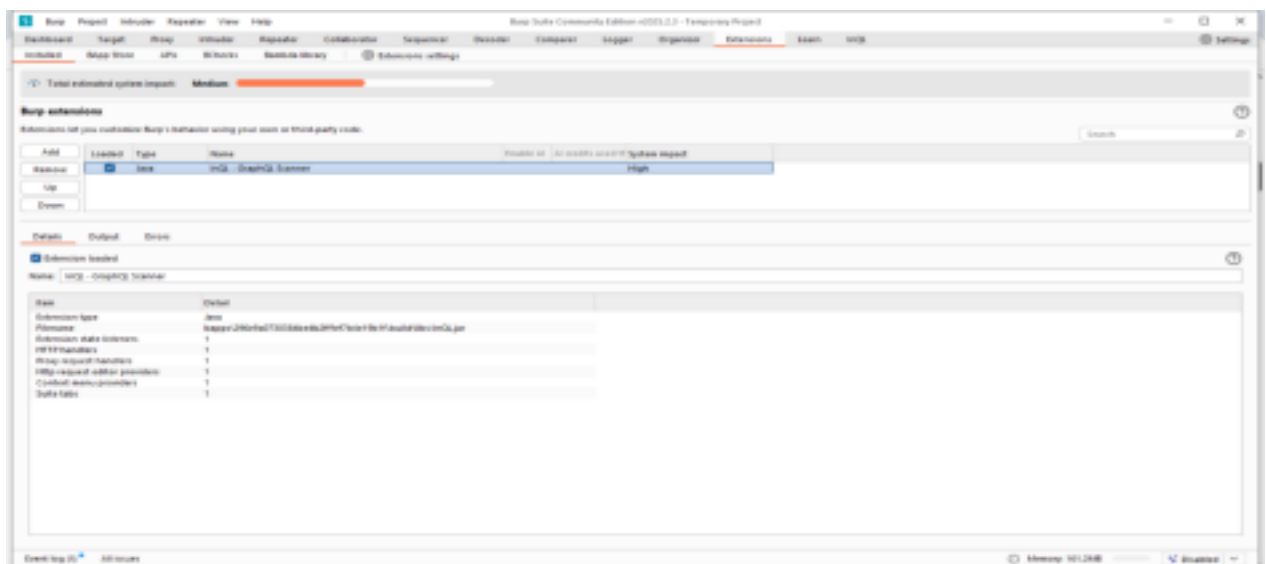


8

Passo 2 –Entre no Burp clique em Extensions, depois em BApp History e no Search digite inql, selecione o inql introspective e clicar no botão instalar.



Passo 3 – O impacto no seu Burp é médio então pode ir em programas instalados e desinstalar caso o seu Burp fique muito lento ou não precise mais utilizá-lo.



9

Passo 4 – Depois no Burp vá até Proxy e clique com o botão direito do mouse em POST /graphql/v1 e em Send to Repeater.



Passo 5 – No Repeater clique em Send, no Response informa o código `getAllBlogsPost`, o campo `data` no response e o campo `POST` no request é um indicativo que a aplicação é vulnerável a acesso através de GraphQL, clicamos na guia `graphql` no request e vemos a estruturação de consulta do GraphQL e substituiremos pelo esquema abaixo:



10

Passo 6 – Ficamos clicando em Send e informa um erro de usuário e senha inválidos e que tem que aguardar um minuto, tanto no Burp quanto na página Web aparecem o mesmo erro.



11

Passo 7 – Podemos utilizar a estrutura abaixo para testar diversos usuários e senhas e vemos que ele responde com OK, mas retorna false no campo success indicando que não é nenhuma daquelas senhas.



Passo 8 – Para automatizar baixe o arquivo em Python através do endereço:
https://github.com/medusa0xf/GraphQL-Rate-Limit-Bypass/blob/master/graphql_gen.py para fazer um ataque de força bruta.



12

Passo 9 – Crie um arquivo chamado wordlists.txt e copie todas as senhas que estão no endereço:
<https://portswigger.net/web-security/authentication/auth-lab-passwords>



Passo 10 – No terminal executar o arquivo `graphql_gen.py` através do comando `python3 graphql_gen.py` e vai gerar o arquivo `mutations.txt`



13

Passo 11 – Abrimos o arquivo `mutations.txt` copiamos todo o seu conteúdo de 100 senhas e colocamos diretamente no request do Burp, no response irá aparecer que na tentativa 98 teve um resultado de `success = true`



Passo 12: Voltamos no arquivo mutations.txt e vemos que no brute force 98 a senha é moon.



14

Passo 13 – Ir em My account e digitar o usuário carlos e a senha moon.



Passo 14 – Depois que fizer o login o laboratório estará concluído.



15

2.4. Análise dos Resultados:

Foi realizada a execução Do Burp Community e alguns comandos no terminal do Kali Linux.

Interpretação dos resultados:

- 1) Na Atividade 1 - Acessando postagens privadas do GraphQL – Instalamos o inql no Burp para ter acesso as consultas, acessamos Proxy HTTP no Burp e enviamos para o Repeater, clicamos em Send no campo Request, verá que não tem o código id 3, copiar a URL e colocar no

inqlScanner, acessamos a primeira query e colocamos no Request dentro da consulta temos maiores informações mas ainda não temos o password, depois acessamos a segunda query , substituímos o id por 3 e conseguimos achar o password.

- 2) Na Atividade 2 – Ignorando proteções de força bruta do GraphQL – Primeiramente tentamos entrar com um login e senha qualquer para que ele informe que a senha está bloqueada por 1 minuto tanto na página web quanto no burp, depois no graphql modificamos a estrutura para um campo de login e senha, depois fazemos um script em python para através de brute force conseguir a senha do carlos, após conseguir a senha da carlos no mutations.txt basta verificar qual resultado informa success true e entrar com as credenciais do carlos, neste caso carlos e na senha moon.

- 3) Abaixo segue o código em python que foi implantado.

```
wordlist_file = 'wordlist.txt' # Replace with the path to your wordlist file
output_file = 'mutations.txt' # The output file where mutations will be written
```

```
# Opening tag of the GraphQL mutation
mutation_start = "mutation {"
```

```
# GraphQL mutation template with an incrementing number
```

```
mutation_template = """
```

```
  bruteforce{}: login(input: {{
    username: "carlos",
    password: "{}"
  }}) {{
    token
    success
  }}
"""
```

```
# Closing tag of the GraphQL mutation
mutation_end = "}"
```

```
# Open the output file for writing
```

16

```
with open(output_file, 'w') as f_out:
```

```
  # Write the opening tag of the mutation
```

```
  f_out.write(mutation_start)
```

```
  f_out.write('\n')
```

```
  # Open the wordlist file for reading
```

```
  with open(wordlist_file, 'r') as f_wordlist:
```

```
    mutation_count = 0
```

```
    for line in f_wordlist:
```

```
      # Remove leading and trailing whitespace from each word
```

```
      password = line.strip()
```

```
      # Generate the mutation with the current password and write it to the output file
```

```
      mutation = mutation_template.format(mutation_count, password)
```

```
      f_out.write(mutation)
```

```
      f_out.write('\n') # Add a newline after each mutation
```



```

mutation_count += 1 # Increment the mutation count for each iteration

# Write the closing tag of the mutation
f_out.write(mutation_end)
f_out.write('\n')

print(f'Mutations have been written to {output_file}')

```

4) Resumo do que o script faz:

Temos que criar um arquivo chamado wordlists.txt com as senhas que são fornecidas no laboratório para realizar o ataque de força bruta.

Depois o python tenta a combinação do usuário carlos com cada senha do arquivo wordlists.txt e coloca no arquivo mutations.txt, depois testamos toda a lista no Burp e quando achamos o arquivo que contém success = true verificamos qual a senha que corresponde e resolvemos o exercício.

2.5. Mitigação:

1. Na Atividade 1 Acessando postagens privadas do GraphQL não deixar as consultas em aberto onde o cliente pode interceptar e encontrar as queries utilizadas e conseguir informações confidenciais.
2. Na Atividade 2 - Ignorando proteções de força bruta do GraphQL não deixar resposta muito visíveis como success = false que saberá quando acertar a senha já que o campo mudará para true .
3. Desabilitar introspecção em API de uso privado.
4. Caso introspecção não possa ser desabilitada, verificar a schema da API para não vazarem campos sensíveis.

17

5. Desabilitar sugestões de campos
6. Limitar o nível de profundidade de uma query
7. Implementar uma análise de custo em uma chamada na API
8. Configurar limites operacionais.

3. Considerações finais

Nesta sala, exploramos técnicas Pentest em Aplicações Web com o GraphQL, onde podemos entrar na aplicação conseguindo uma senha através de um post privado e conseguindo o usuário e senha através de força bruta.

As principais lições aprendidas incluem:

- 1) Não deixar consultas expostas que possam dar dicas de como contornar a sua aplicação GraphQL.
- 2) Não autorizar o cliente fazer consultas diretamente na sua aplicação.
- 3) Não permitir scripts que possam manipular o comportamento esperado.
- 4) Autorizar consultas simples que não faça o aplicativo fornecer mais informações do que o necessário.

5) Verificar as chamadas de API e configurar limites para que o aplicativo funcione corretamente.

4. Referências

Site: <https://portswigger.net/web-security/graphql/lab-graphql-reading-private-posts> Site:

<https://portswigger.net/web-security/graphql/lab-graphql-brute-force-protection-bypass>

Site: <https://www.youtube.com/watch?v=r-O5aok8TeM>

Site:

https://github.com/medusa0xf/GraphQL-Rate-Limit-Bypass/blob/master/graphql_gen.py

Site: https://www.youtube.com/watch?v=RzLzMH_Mu7I